

XOR-Prime

Hard Cryptography Challenge - Writeup

*"They asked me to generate two RSA keys in a low entropy environment.
I came up with something that only needs half of the usual entropy!
I doubt it will be possible but try to break it anyway..."*

Category: Cryptography

Difficulty: Hard

Flag: CSCPT{d0nt_us3_rel4ted_pr1mes}

1. Background: What is RSA?

RSA is one of the most widely used public-key cryptosystems. It relies on the mathematical difficulty of factoring the product of two large prime numbers. Here is a simplified overview:

1.1 Key Generation

1. Choose two large, random, distinct prime numbers p and q (typically 512 bits each).
2. Compute $N = p * q$. This is the public modulus (1024 bits).
3. Compute the totient: $\phi(N) = (p - 1) * (q - 1)$.
4. Choose a public exponent e (commonly 65537).
5. Compute the private exponent d such that $e * d = 1 \pmod{\phi(N)}$.
6. Public key: (N, e) . Private key: d .

1.2 Encryption and Decryption

To encrypt a message m : $c = m^e \pmod{N}$

To decrypt a ciphertext c : $m = c^d \pmod{N}$

The security of RSA depends entirely on the fact that an attacker who only knows N cannot efficiently find p and q . If p and q are found, the attacker can compute d and decrypt any message.

1.3 Why Primes Must Be Independent

A critical requirement of RSA is that each key pair uses independently generated primes. If multiple RSA keys share factors, or if the primes are mathematically related, an attacker can exploit these relationships to recover the factors. This challenge demonstrates exactly such a flaw.

2. Challenge Analysis

2.1 The Source Code

We are given the following Python script (gen_keys.py):

```
from Crypto.Util.number import *
from pwn import xor
from flag import FLAG

p1 = getPrime(512)
q1 = getPrime(512)
BITS = 2**512

def generate_prime(p, q, start):
    i = start
    while 1:
        i += 1
        r = (pow(p, i, BITS) ^ pow(q, i, BITS)) + 1
        if isPrime(r):
            return r, i

p2, k = generate_prime(p1, q1, 0)
q2, l = generate_prime(p1, q1, k+1)

N1 = p1*q1
N2 = p2*q2
e = 65537

print('N1 =', N1)
print('N2 =', N2)
print('c =', pow(FLAG, e, N1))
print('And its fast too, only took', l, 'iterations')
```

2.2 The Output

And we are given the output (output.txt) with the following values:

```
N1 = 7713706983181812380724077445595076276799...(truncated)
N2 = 5525732987607991393746708411247537346817...(truncated)
c = 5895927874393764585159138201376040175359...(truncated)
And its fast too, only took 228 iterations in total
```

2.3 Understanding the Vulnerability

The challenge description says: "only needs half of the usual entropy." This is the key hint. Normally, two

RSA keys require four independently random primes (p_1 , q_1 , p_2 , q_2). But here, only p_1 and q_1 are randomly generated. The primes p_2 and q_2 are deterministically derived from p_1 and q_1 using the formula:

$$r = (\text{pow}(p_1, i, 2^{512}) \wedge \text{pow}(q_1, i, 2^{512})) + 1$$

This means p_2 and q_2 are completely determined by p_1 and q_1 . There is a mathematical relationship between the two RSA moduli N_1 and N_2 . We can exploit this relationship to recover p_1 and q_1 , and then decrypt the flag.

3. The Key Insight: XOR is Bitwise

The XOR (exclusive or) operation works bit by bit. This means that bit j of the result $(p_1^i \text{ XOR } q_1^i)$ only depends on bits 0 through j of the inputs. Similarly, modular exponentiation $\text{mod } 2^b$ only depends on the lowest b bits of the base.

3.1 What Does This Mean?

If we know the lowest b bits of p_1 , we can:

1. Compute the lowest b bits of q_1 , because $p_1 * q_1 = N_1$, so:

$$q_1 \text{ mod } 2^b = N_1 * (p_1)^{(-1)} \text{ mod } 2^b$$

2. Compute the lowest b bits of p_2 and q_2 , because:

$$p_2 \text{ mod } 2^b = (p_1^k \text{ mod } 2^b \text{ XOR } q_1^k \text{ mod } 2^b) + 1 \text{ mod } 2^b$$

$$q_2 \text{ mod } 2^b = (p_1^{228} \text{ mod } 2^b \text{ XOR } q_1^{228} \text{ mod } 2^b) + 1 \text{ mod } 2^b$$

3. Check if $p_2 * q_2 = N_2 \text{ mod } 2^b$

This lets us build up p_1 one bit at a time, from the least significant bit to the most significant bit. At each step, we try both possibilities (0 or 1 for the next bit) and keep only the ones that satisfy ALL constraints.

3.2 This Technique Is Called "2-adic Lifting"

This technique is related to Hensel's Lemma, a classical result in number theory. It works whenever equations involve operations that are "local" in the 2-adic sense -- meaning that the low bits of the output depend only on the low bits of the input. Both multiplication $\text{mod } 2^b$ and XOR have this property, which is why the attack works.

4. The Attack, Step by Step

Step 1: Determine the Iteration Indices k and l

From the output, we know $l = 228$ (total iterations). This means q_2 was generated at iteration $i = 228$. We do NOT know k (the iteration for p_2). However, k must be between 1 and 226 (since q_2 starts searching at $k+2$ and finishes at 228).

Our strategy: try each possible k value (1 to 226) and see which one allows the bit-by-bit recovery to succeed (i.e., candidates survive all 512 bit levels).

Step 2: Initialize the Bit-by-Bit Search

We know that p_1 is an odd prime, so its least significant bit is 1. We start with:

$$\text{candidates} = \{1\} \quad (p_1 \bmod 2 = 1)$$

Then for each bit position b from 2 to 512:

Step 3: Extend Each Candidate

For each candidate $p1_low$ (which represents $p_1 \bmod 2^{(b-1)}$), we try two extensions:

- $p1_try = p1_low$ (new bit = 0)
- $p1_try = p1_low + 2^{(b-1)}$ (new bit = 1)

For each $p1_try$, we compute $q1_try = N_1 * p1_try^{(-1)} \bmod 2^b$.

Step 4: Apply the N_2 Constraint

We compute:

$$\begin{aligned} p2_mod &= (p1_try^k \text{ XOR } q1_try^k) + 1 \bmod 2^b \\ q2_mod &= (p1_try^{228} \text{ XOR } q1_try^{228}) + 1 \bmod 2^b \end{aligned}$$

Then check if $p2_mod * q2_mod = N_2 \bmod 2^b$. If yes, keep this candidate. If no, discard it.

This constraint eliminates roughly half the candidates at each bit level, preventing exponential blowup.

Step 5: Final Verification

After processing all 512 bits, we have a set of candidates for $p_1 \bmod 2^{512}$. Since p_1 is a 512-bit prime, $p_1 \bmod 2^{512} = p_1$ itself. We check which candidates actually divide N_1 . The one that does is the real p_1 .

5. Complete Solution Code

Below is the complete, working exploit script (solve_final.py):

```
from Crypto.Util.number import long_to_bytes

N1 = 77137069831818123807240774455950762767990413325793
... # (full value in solve_final.py)
N2 = 55257329876079913937467084112475373468177837748963
... # (full value in solve_final.py)
c = 58959278743937645851591382013760401753597451260953
... # (full value in solve_final.py)
e = 65537
BITS_MOD = 2**512
k_val = 210 # found by scanning
l_val = 228 # given in the output

candidates = {1} # p1 is odd
for b in range(2, 513):
    mod = 1 << b
    n1_mod = N1 % mod
    n2_mod = N2 % mod
    new_candidates = set()
    for p1_low in candidates:
        for bit in [0, 1]:
            p1_try = p1_low | (bit << (b - 1))
            q1_try = (n1_mod * pow(p1_try, -1, mod)) % mod
            p2 = ((pow(p1_try, k_val, mod) ^
                    pow(q1_try, k_val, mod)) + 1) % mod
            q2 = ((pow(p1_try, l_val, mod) ^
                    pow(q1_try, l_val, mod)) + 1) % mod
            if (p2 * q2) % mod == n2_mod:
                new_candidates.add(p1_try)
    candidates = new_candidates

for p1 in candidates:
    if N1 % p1 == 0:
        q1 = N1 // p1
        phi = (p1 - 1) * (q1 - 1)
        d = pow(e, -1, phi)
        flag = long_to_bytes(pow(c, d, N1))
        print(flag)
# b'CSCPT{d0nt_us3_rel4ted_pr1mes}'
```

5.1 Finding k = 210

To find k , we run the bit-by-bit search for each candidate k from 1 to 226. For wrong values of k , the constraint $p^2 \cdot q^2 = N^2 \bmod 2^b$ will become inconsistent early on, and all candidates will be eliminated. For $k = 210$, candidates survive all 512 levels, confirming it is the correct value. This scanning step takes a few minutes but is fully automated.

6. Why This Matters: Lessons Learned

6.1 Never Reuse Entropy Across Keys

The fundamental flaw is that two RSA keys were generated using the same source of randomness (p_1 and q_1). The primes p_2 and q_2 are entirely determined by p_1 and q_1 . This creates a mathematical link between N_1 and N_2 that should never exist.

In real-world cryptography, every prime must be generated from an independent, high-quality entropy source. Standards like NIST SP 800-90A specify how to properly seed random number generators for key generation.

6.2 XOR Does Not Provide Cryptographic Independence

The author tried to create "new" primes by XOR-ing powers of existing primes. While XOR is used extensively in cryptography (e.g., in stream ciphers and block ciphers), it does not magically create independent values. The bit-by-bit structure of XOR is precisely what allows the 2-adic lifting attack.

6.3 Modular Arithmetic mod 2^n is Weak

Working modulo 2^{512} (a power of 2) is inherently weaker than working modulo a prime. Powers of 2 have a very special algebraic structure that makes them vulnerable to bit-by-bit attacks. This is why most serious cryptographic operations use prime moduli.

7. Summary

This challenge featured an RSA system where two key pairs shared related primes. The primes of the second key were derived from the first key's primes using XOR of modular exponentiations. Because XOR is a bitwise operation, we could recover the original primes one bit at a time using a technique called 2-adic lifting.

The attack required:

1. Understanding the bitwise nature of XOR
2. Recognizing that mod 2^b arithmetic is "local" (low bits only depend on low bits)
3. Building p_1 from LSB to MSB, pruning with both N_1 and N_2 constraints
4. Scanning for the unknown iteration index k

Runtime: ~5 minutes on a modern laptop.

Flag: CSCPT{d0nt_us3_rel4ted_pr1mes}